

- Use $\text{\LaTeX}$ to write-up your solutions (in the solution blocks of the source $\text{\LaTeX}$ file of this assignment), and submit the resulting single pdf file at GradeScope by the suggested due date. Within GradeScope, indicate the page number where your solution to each question starts, else we won't be able to grade it!
(**Note:** Due to the COVID situation and related institute mandates, there won't be late submission penalties to give students with poor internet access or other difficulties flexibility in completing assignments. If you do have proper internet access and other facilities, we highly recommend you to submit your solutions by the suggested due date to avoid being overwhelmed by all your works when institute reopens.)
- Collaboration is encouraged, but all write-ups must be done individually and independently, and mention your collaborator(s) if any. Same rules apply for codes written for any programming assignments (i.e., write your own code; we will run plagiarism checks on codes).
- If you have referred a book or any other online material for obtaining a solution, please cite the source. Again don't copy the source *as is* - you may use the source to understand the solution, but write-up the solution in your own words.

1. (14 points) [CIRCULAR CLEANUP!] [from Stanford's Stat375 course]

We want to implement a simple image segmentation/denoising algorithm based on Cluster Graph Belief Propagation (aka CG-BP or Loopy-BP) inference. The algorithm will take as input a $n \times n$ binary image encoded by the $0 - 1$ matrix:

$$\underline{y} = \{y_i \in \{0, 1\} : i = (i_1, i_2) \in [n] \times [n]\},$$

and return a $0 - 1$ matrix:

$$\underline{\hat{x}} = \{\hat{x}_i \in \{0, 1\} : i = (i_1, i_2) \in [n] \times [n]\}.$$

We expect nearby entries in $\hat{x}$ to have similar values; so to accomplish this goal, we introduce a graphical model of the form:

$$\mu(\underline{x}, \underline{y}) = \frac{1}{Z} \left(\prod_{i \in [n-1] \times [n]} \psi(x_i, x_{i+\hat{d}}) \right) \left(\prod_{i \in [n] \times [n-1]} \psi(x_i, x_{i+\hat{r}}) \right) \left(\prod_{i \in [n] \times [n]} \phi(x_i, y_i) \right),$$

where for $i = (i_1, i_2)$, we let $i + \hat{d} = (i_1 + 1, i_2)$ and $i + \hat{r} = (i_1, i_2 + 1)$. The compatibility functions are given in matrix form by:

$$\psi(\cdot, \cdot) = \begin{pmatrix} 1 + \theta & 1 - \theta \\ 1 - \theta & 1 + \theta \end{pmatrix}, \text{ and}$$

$$\phi(\cdot, \cdot) = \begin{pmatrix} 1 + \gamma & 1 - \gamma \\ 1 - \gamma & 1 + \gamma \end{pmatrix}$$

for some $\theta \in [0, 1]$ and $\gamma \in [0, 1]$.

- (a) (3 points) Write the CG-BP (sum-product and max-product) message update rules of this model, assuming a Bethe Cluster Graph (Factor Graph) representation.

Solution: Assuming a Bethe Cluster Graph representation with hidden variables $\underline{x}$ as nodes and adjacent pairs of hidden variables making up factors (factors denoted by X_a and nodes by x_i), the message update rules are as follows:

$$m_{i \rightarrow a} = \prod_{k \in \text{Nbrs}(i) \setminus a} m_{k \rightarrow i}(x_i) \quad (1a)$$

$$m_{a \rightarrow i} = \sum_{\text{scope}(X_a) \setminus x_i} \psi(X_a) \prod_{k \in \text{Nbrs}(a) \setminus i} m_{k \rightarrow a}(x_k) \quad (1b)$$

The marginal at a node,

$$\beta(x_i) \propto \prod_{k \in \text{Nbrs}(i)} m_{k \rightarrow i}(x_i) \quad (1c)$$

Note that in Equation 1a and Equation 1c, $\text{Nbrs}(i)$ also contains the observable node y_i , and the message from y_i to x_i :

$$m_{y_i \rightarrow x_i} = \phi(x_i, y_i) \delta(y_i) \quad (1d)$$

For max-product, we only change the summation to max:

$$m_{i \rightarrow a} = \prod_{k \in \text{Nbrs}(i) \setminus a} m_{k \rightarrow i}(x_i) \quad (1e)$$

$$m_{a \rightarrow i} = \max_{\text{scope}(X_a) \setminus x_i} \psi(X_a) \prod_{k \in \text{Nbrs}(a) \setminus i} m_{k \rightarrow a}(x_k) \quad (1f)$$

- (b) (2 points) Provide a concise pseudocode that specifically shows how to initialize and schedule the message updates from above, in order to achieve the goal of constructing a denoised image $\hat{x}$ from a noisy image y .

Solution: The pseudocode for synchronous (or parallel) loopy belief propagation is as follows (Note that here I have taken an approach where the message from visible to hidden node $m_{y_i \rightarrow x_i}$ is taken as the prior belief of the hidden node. Implementation remains the same as previous question):

```

Procedure Loopy Belief Propagation(
    n // The size of the grid
    N // Number of iterations
     $\psi$  // compatibility between hidden variables
     $\phi$  // compatibility between hidden and visible variables
    x // hidden variables
    y // visible variable (binary image)
)

1:   Initialize clique_messages = ones(n, n, 2, 4)
2:   Initialize factor_messages = ones(n, n, 2, 4)
3:   priors = ones(n, n, 2)      # Prior beliefs at the node or  $m_{y_i \rightarrow x_i}$ 
4:
5:   for j = 1 to  $n^2$ :
6:       Update priors as in Equation 1d
7:
8:   for iterations = 1 to N:
9:       for all  $x_i$  (parallel phase):
10:          send factor_messages  $m_{i \rightarrow a}(x_i)$  as in Equation 1a
11:          normalize factor_messages
12:          send clique_messages  $m_{a \rightarrow i}(x_i)$  as in Equation 1b
13:          normalize clique_messages
14:       end for
15:   end for
16:
17:   Find marginal  $\beta(x_i)$  at node i as in Equation 1c
18:    $x_i = \text{argmax } \beta(x_i)$ 

```

In practice the parallel message passing phase is done in 4 steps (right, left, top, bottom). For sequential message passing the **step 9** would loop over the all nodes one by one, which is considerably slower. Both types are implemented in the code, available at <https://>

github.com/parthnatekar/Loopy-Belief-Propagation. Please refer to the code for more details.

- (c) (5 points) Implement the pseudocode and test your program on a noisy image $\underline{y}$ obtained as follows: construct a 100×100 binary matrix $\underline{x}$ corresponding to a circle of radius 25 with center in position (50,50), and add noise by flipping each entry independently with probability p to obtain $\underline{y}$. Run the program on three images generated with $p = 0.1, 0.2$, and 0.3 . Display the corresponding images (input and output), together with the values of parameters (θ, γ) used. (Note: Please also provide your denoising code that takes input/output images as comma-separated ASCII files with three columns $(i_1, i_2, \text{value}_{(i_1, i_2)})$ to help check your work).

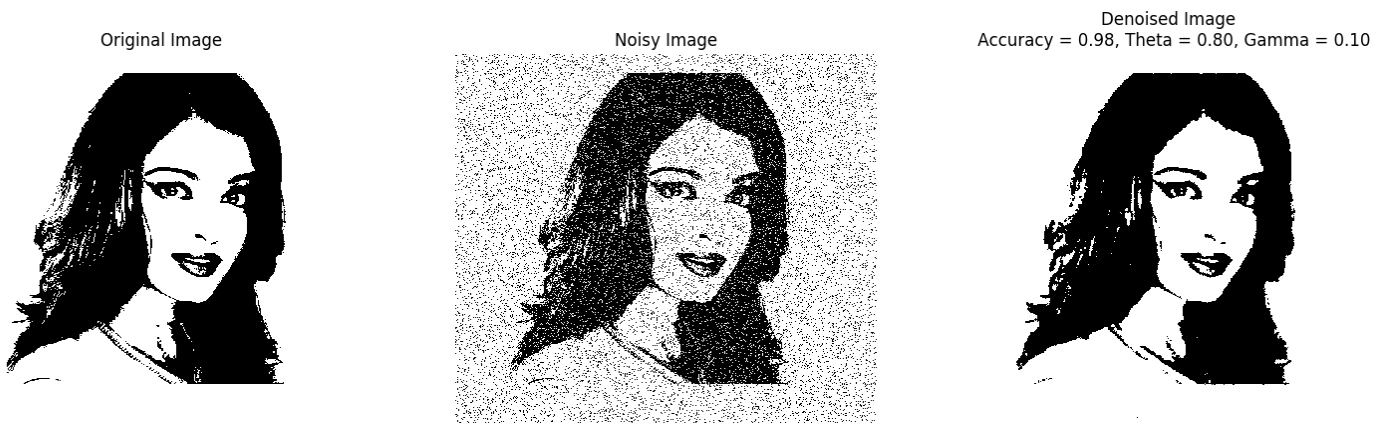


Figure 1: Result of Algorithm on image of Aishwarya Rai (Because why not)

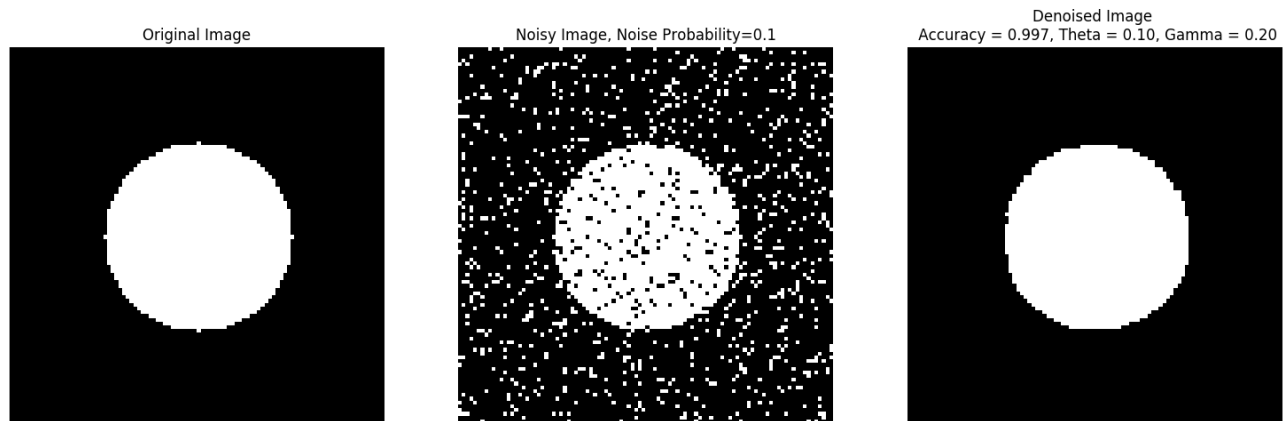


Figure 2: Result of Algorithm on adding noise with $p = 0.1$ (At optimal parameter values from next question)

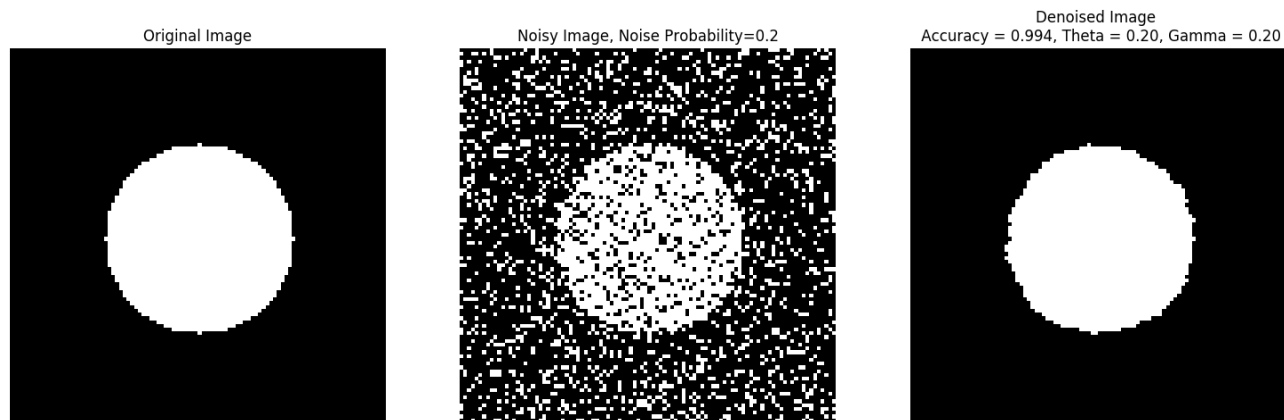


Figure 3: Result of Algorithm on adding noise with $p = 0.2$

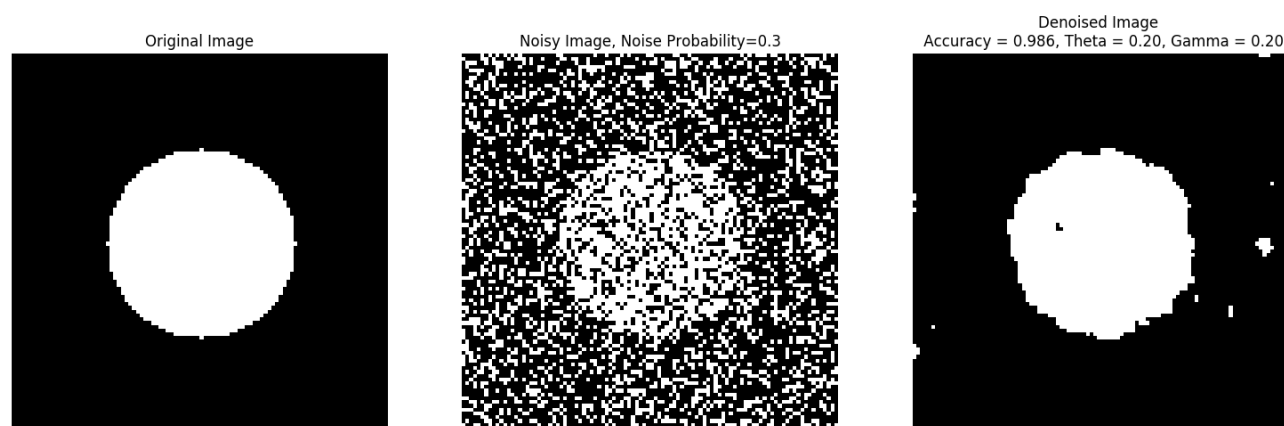


Figure 4: Result of Algorithm on adding noise with $p = 0.3$

Solution:

Below is the function for synchronous loopy belief propagation on a given input image. Full code is also available at above link and has been submitted.

For running the code, run

```
python3 loopy_bp.py -p PATH TO IMAGE/CSV -m MODE (sync or seq)
-n NOISE (Whether to add noise to image, True or False)
```

```
def messages_sync(self):
```

```
    clique_messages = np.ones((self.height, self.width, 2, 4))
```

```

factor_messages = np.ones((self.height, self.width, 2, 4))
factor_norm = np.ones((self.height, self.width, 1, 4))
clique_norm = np.ones((self.height, self.width, 1, 4))
beta = np.ones((self.height, self.width, 2))

for j in range(self.height*self.width):
    beta[np.unravel_index(j, (self.height, self.width))] = self.compatibility_inter[
        (self.height, self.width))]

for i in range(self.iterations):

    # Calculate factor messages for grid
    factor_messages[:, :, :, 0] *= beta*clique_messages[:, :, :, 0]*\
        clique_messages[:, :, :, 2]*\
        clique_messages[:, :, :, 3]
    factor_messages[:, :, :, 1] *= beta*clique_messages[:, :, :, 1]*\
        clique_messages[:, :, :, 2]*\
        clique_messages[:, :, :, 3]
    factor_messages[:, :, :, 2] *= beta*clique_messages[:, :, :, 2]*\
        clique_messages[:, :, :, 0]*\
        clique_messages[:, :, :, 1]
    factor_messages[:, :, :, 3] *= beta*clique_messages[:, :, :, 3]*\
        clique_messages[:, :, :, 0]*\
        clique_messages[:, :, :, 1]

    for j in range(clique_messages.shape[-1]):
        factor_norm[:, :, :, j] = np.sum(factor_messages[:, :, :, j],
            axis=2)[..., None]

    # Normalize factor messages
    for j in range(clique_messages.shape[-1]):
        for ii in range(2):
            factor_messages[:, :, ii, j] /= \
                np.squeeze(factor_norm[:, :, :, j])

    # Calculate clique messages for grid
    clique_messages[:, :, :, 0] *= self.compatibility_inter.dot\
        (np.append(np.ones((self.height, 1, 2), axis=1),
            factor_messages[:, :, -1, :, 0], axis=1)).

```

```

        clique_messages[:, :, :, 1] *= self.compatibility_inter.dot\
            (np.append(factor_messages[:, 1:, :],
                        np.ones((self.height, self.width, 2))),
            axis=-1)
        clique_messages[:, :, :, 2] *= self.compatibility_inter.dot\
            (np.append(factor_messages[1: :, :, :],
                        np.ones((self.height, self.width, 2))),
            axis=-1)
        clique_messages[:, :, :, 3] *= self.compatibility_inter.dot\
            (np.append(np.ones((1, self.width, 2)),
                        factor_messages[0, :, :]),
            axis=-1)

    for j in range(clique_messages.shape[-1]):
        clique_norm[:, :, :, j] = np.sum(clique_messages[:, :, :, j], axis=-1)

    # Normalize clique messages
    for j in range(clique_messages.shape[-1]):
        for ii in range(2):
            clique_messages[:, :, ii, j] /= np.squeeze(clique_norm[:, :, :, j])

    # Calculate marginals at each node
    for j in range(clique_messages.shape[-1]):
        beta *= np.append(np.ones((self.height, 1, 2)), factor_messages[:, :, :, j],
                           *np.append(np.ones((1, self.width, 2)), factor_messages[0, :, :, j],
                           *np.append(factor_messages[1: :, :, j], np.ones((self.height, self.width, 2))),
                           *np.append(factor_messages[:, 1: :, j], np.ones((self.height, 1, 2))))

    # Check accuracy
    if self.add_noise:
        accuracy = np.sum(self.noiseless_image == np.argmax(beta, axis = 2))
        return(self.image, np.argmax(beta, axis = 2), self.noiseless_image, accuracy)
    else:
        return(self.image, np.argmax(beta, axis = 2))

```

- (d) (4 points) Count the number of positions at which the estimated $\hat{x}$ matches the noise-free circle x , and average this number across many input instances/runs of your program. Report this average performance for a few values of (θ, γ) , and comment on which choice offered the best performance. If you wanted to be more systematic to find the best value of (θ, γ) among all values it takes (instead of trying only a few values and taking its best), briefly mention what approach would you take?

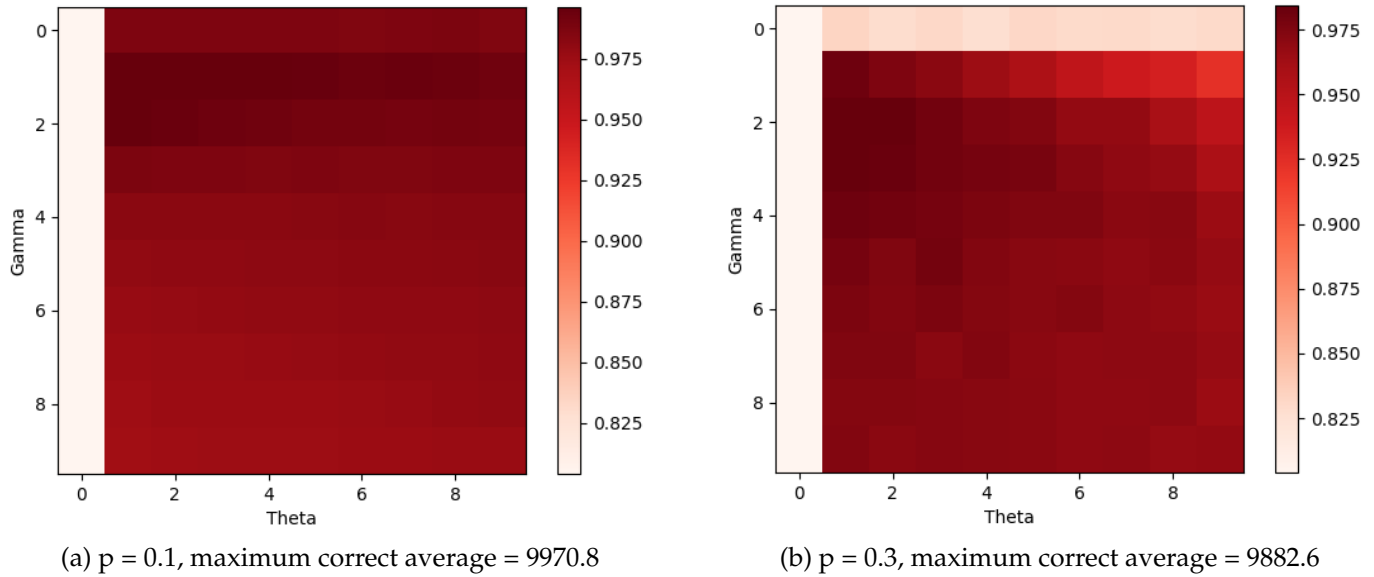


Figure 5: Average Accuracy over 10 runs of the algorithm for $p = 0.1, 0.3$. Darker means greater accuracy. Gamma and Theta values are axis-value divided by 10

Solution: The heat-map for average number of correct pixels over 10 runs is shown in Figure 5 for $p = 0.1, 0.3$ (Count values are divided by 10,000 to show accuracy, while parameter values are axis value divided by 10). For example, from the heatmap, $\theta \approx 0.1$ and $\gamma \approx 0.2$ gives the highest average number of correct pixels for $p = 0.1$. For these values, the average number of correct pixels is 9970.8 (Accuracy = 99.7%).

A systematic approach would be to take the expectation of the accuracy using a grid search as above, or perform discrete optimization using an approach like a genetic algorithm.

2. (12 points) [APPROXIMATE INFERENCE ON RBMs] Consider a Markov network over m hidden $\{0, 1\}$ variables H_i and n visible $\{0, 1\}$ -variables V_j . Let the joint distribution of this model be defined as:

$$P(H = h, V = v) = \frac{1}{Z} \exp(h^T W v + c^T v + b^T h),$$

where $W \in \mathbb{R}_{m,n}$, $c \in \mathbb{R}_n$, $b \in \mathbb{R}_m$ are the parameters of the model.

This is called a *Restricted Boltzmann Machine* or RBM since the edges in the Markov network are restricted to be between a hidden node and a visible node (and specifically no edges are allowed between any two hidden nodes or any two visible nodes; note that this property makes RBMs, specifically stacking of multiple RBMs, have interesting links to generative deep learning models with varied applications). To do approximate inference on a RBM model, we could employ any of these methods. Write down their update equations and simplify your terms as much as possible.

- (a) (3 points) Derive standard Gibbs sampling updates of this RBM model (specifically, provide two update equations, one for sampling each H_i given all other variables and another for sampling V_j given all other variables). Report the running time complexity per update and per epoch (an epoch comprises update of every variable in the model).

Solution: The joint distribution is as follows:

$$P(V, H) = \frac{1}{Z} \prod_i \prod_j e^{w_{ij} h_i v_j} \prod_i e^{b_i h_i} \prod_j e^{c_j v_j}$$

Standard Gibbs sampling updates for RBM:

$$P(H_k = h_k | H_{\neg k}, V) = \frac{P(H_k = h_k, V, H_{\neg k})}{P(H_{\neg k}, V)} \quad (2a)$$

$$= \frac{\prod_j e^{c_j v_j} \prod_i e^{b_i h_i} \prod_{i,j} e^{w_{ij} h_i v_j}}{\sum_{H_k} \prod_j e^{c_j v_j} \prod_i e^{b_i h_i} \prod_{i,j} e^{w_{ij} h_i v_j}} \quad (2b)$$

$$= \frac{\left(\prod_j e^{c_j v_j} \prod_{i \setminus k} e^{b_i h_i} \prod_{i \setminus k, j} e^{w_{ij} h_i v_j} \right) e^{b_k h_k + \sum_j w_{kj} h_k v_j}}{\left(\prod_j e^{c_j v_j} \prod_{i \setminus k} e^{b_i h_i} \prod_{i \setminus k, j} e^{w_{ij} h_i v_j} \right) \sum_{H_k} e^{b_k h_k + \sum_j w_{kj} h_k v_j}} \quad (2c)$$

$$= \frac{e^{b_k h_k + \sum_j w_{kj} h_k v_j}}{\sum_{H_k} e^{b_k h_k + \sum_j w_{kj} h_k v_j}} \quad (2d)$$

$$= \frac{e^{b_k h_k + \sum_j w_{kj} h_k v_j}}{1 + e^{b_k + \sum_j w_{kj} v_j}} \quad (2e)$$

$$P(H_k = 1 | H_{\neg k}, V) = \frac{1}{1 + e^{-(b_k + \sum_j w_{kj} v_j)}} \quad (2f)$$

$$= \sigma(b_k + \sum_j w_{kj} v_j) \quad (2g)$$

And,

$$P(H_k = 0 | H_{\neg k}, V) = 1 - \sigma(b_k + \sum_j w_{kj} v_j) \quad (2h)$$

Similarly,

$$P(V_k = 1 \mid H, V_{-k}) = \sigma(c_k + \sum_i w_{ik} h_i) \quad (2i)$$

$$P(V_k = 0 \mid H, V_{-k}) = 1 - \sigma(c_k + \sum_i w_{ik} h_i) \quad (2j)$$

The running time complexity per update is $\mathcal{O}(n)$ for hidden variables and $\mathcal{O}(m)$ for visible variables.

Per epoch, there are n updates of cost m and vice-versa. Thus, the running time per epoch is $\mathcal{O}(2mn) \approx \mathcal{O}(mn)$

- (b) (3 points) Derive Block Gibbs sampling updates of this RBM model. Consider two blocks (one for all hidden node, and another for all visible nodes). Report the running time per block update - can you make this update as efficient as possible (i.e., make the running time per epoch the same as one epoch of standard Gibbs sampling) by exploiting the restricted structure of the RBM network? Can you parallelize each block update and what will be the resulting running time?

Solution:

$$P(H \mid V) = \frac{P(V, H)}{P(V)} \quad (3a)$$

$$= \frac{\frac{1}{Z} \exp[-E(V, H)]}{\frac{1}{Z} \sum_H \exp[-E(V, H)]} \quad (3b)$$

For a naive implementation, the runtime per block would be $\mathcal{O}(2^m)$ for hidden units and $\mathcal{O}(2^n)$ for visible units. This is because of the summation in the denominator which is a summation over both values of all hidden variables. However, the complexity can be greatly reduced by using the restricted structure.

Consider the denominator,

$$\frac{1}{Z} \sum_H \exp(-E(V, H)) = \frac{1}{Z} \prod_j e^{c_j v_j} \prod_i \left(\sum_{h_i} e^{(b_i h_i + \sum_j w_{ij} h_i v_j)} \right) \quad (3c)$$

$$= \frac{1}{Z} \prod_j e^{c_j v_j} \prod_i \left(1 + e^{(b_i + \sum_j w_{ij} v_j)} \right) \quad (3d)$$

$$\therefore P(H | V) = \frac{\prod_j e^{c_j v_j} \prod_i e^{(b_i h_i + \sum_j w_{ij} h_i v_j)}}{\prod_j e^{c_j v_j} \prod_i (1 + e^{(b_i + \sum_j w_{ij} v_j)})} \quad (3e)$$

$$= \prod_i \frac{e^{(b_i h_i + \sum_j w_{ij} h_i v_j)}}{1 + e^{(b_i + \sum_j w_{ij} v_j)}} \quad (3f)$$

Thus, similar to the standard Gibbs sampling update, each hidden unit is randomly sampled to be 1 with probability $\sigma(b_k + \sum_j w_{kj} v_j)$. The difference in Block Gibbs is that the all hidden units are sampled simultaneously.

Similarly,

$$P(V | H) = \prod_j \frac{e^{(c_j v_j + \sum_i w_{ij} h_i v_j)}}{1 + e^{(c_j + \sum_i w_{ij} h_i)}} \quad (3g)$$

After exploiting the restricted structure, the running time per block update is $\mathcal{O}(mn)$, and hence the running time per epoch is $\mathcal{O}(2mn)$. After parallelizing each block update, the running time will be $\mathcal{O}(m + n)$, since all summations are computed in parallel.

- (c) (3 points) Derive the Naïve Mean Field updates of this RBM model. Describe how you can use these updates (after convergence to their fixed-point values) to approximate, specifically lower-bound, the partition function Z ?

Solution: Assuming $Q(V, H)$ is the approximate distribution,

We want to minimize,

$$KL(Q \parallel P) = \sum_{V, H} Q(V, H) \log\left(\frac{Q(V, H)}{P(V, H)}\right)$$

Assuming a naive-mean field approximation for $Q(V, H)$,

$$Q(V, H) = \prod_i q_i(h_i) \prod_j q_j(v_j)$$

Then,

$$KL(Q \parallel P) = \sum_{V, H} Q(V, H) \log [Q(V, H) - P(V, H)] \quad (4a)$$

$$= \left(\sum_{V,H} \prod_i q_i(h_i) \prod_j q_j(v_j) \log \left[\prod_i q_i(h_i) \prod_j q_j(v_j) - \prod_i e^{b_i h_i} \prod_j e^{c_j v_j} \prod_{i,j} w_{ij} h_i v_j \right] \right) + \sum_{V,H} Q(V,H) \log(Z) \quad (4b)$$

$$= \left(\sum_{V,H} \prod_i q_i(h_i) \prod_j q_j(v_j) \log \left[\prod_i q_i(h_i) \prod_j q_j(v_j) - \prod_i e^{b_i h_i} \prod_j e^{c_j v_j} \prod_{i,j} w_{ij} h_i v_j \right] \right) + \log(Z) \quad (4c)$$

Since $\log(Z)$ is a constant w.r.t the variational parameters, we can minimize KL-Divergence by minimizing the Variational Free Energy $F = \text{KL}(Q \parallel P) - \log(Z)$ (Note that here F is defined negative of the Free Energy in the course notes i.e. $F = -H(Q) - E_Q \log(P'(V,H))$. The final computation is the same).

$$F = \sum_H \sum_i q_i(h_i) \log(q_i(h_i)) + \sum_V \sum_j q_j(v_j) \log(q_j(v_j)) - \sum_{V,H} \sum_i q_i(h_i) b_i h_i \sum_j q_j(v_j) - \sum_{V,H} \sum_j q_j(v_j) c_j v_j \sum_i q_i(h_i) - \sum_{V,H} \sum_{i,j} w_{ij} h_i v_j q_i(h_i) q_j(v_j) \quad (4d)$$

Let $q_i(h_i = 1) = \mu_i$ and $q_j(v_j = 1) = \mu_j$. Then,

$$F = \sum_i \mu_i \log(\mu_i) + (1 - \mu_i) \log(1 - \mu_i) + \sum_j \mu_j \log(\mu_j) + (1 - \mu_j) \log(1 - \mu_j) - \sum_H \sum_i q_i(h_i) b_i h_i \sum_j \sum_V q_j(v_j) - \sum_V \sum_j q_j(v_j) c_j v_j \sum_i \sum_H q_i(h_i) - \sum_{i,j} w_{ij} \sum_{V,H} h_i v_j q_i(h_i) q_j(v_j) \quad (4e)$$

$$= \sum_i \mu_i \log(\mu_i) + (1 - \mu_i) \log(1 - \mu_i) + \sum_j \mu_j \log(\mu_j) + (1 - \mu_j) \log(1 - \mu_j) - \sum_H \sum_i q_i(h_i) b_i h_i - \sum_V \sum_j q_j(v_j) c_j v_j - \sum_{i,j} w_{ij} \sum_{V,H} h_i v_j q_i(h_i) q_j(v_j) \quad (4f)$$

$$= \sum_i \left(\mu_i \log(\mu_i) + (1 - \mu_i) \log(1 - \mu_i) \right) + \sum_j \left(\mu_j \log(\mu_j) + (1 - \mu_j) \log(1 - \mu_j) \right) - \sum_i \mu_i b_i - \sum_j \mu_j c_j - \sum_{i,j} w_{ij} \mu_i \mu_j \quad (4g)$$

Then, minimizing $\log(F)$ w.r.t. the parameters μ_i and μ_j , we get:

$$\mu_i = \sigma \left(\sum_j w_{ij} \mu_j + b_i \right) \quad (4h)$$

And,

$$\mu_j = \sigma \left(\sum_i w_{ij} \mu_i + c_j \right) \quad (4i)$$

The variational mean field approximation yields an upper bound on the partition function Z .

$$\log(Z) = KL - F \quad (4j)$$

As KL-Divergence is greater than zero, then, at equilibrium

$$\log(Z) \geq -F \quad (4k)$$

$$\Rightarrow \log Z \geq \min_{\mu_i, \mu_j} - \left(\sum_i \mu_i \log(\mu_i) + (1 - \mu_i) \log(1 - \mu_i) + \sum_j \mu_j \log(\mu_j) + (1 - \mu_j) \log(1 - \mu_j) - \sum_i \mu_i b_i - \sum_j \mu_j c_j - \sum_{i,j} w_{ij} \mu_i \mu_j \right) \quad (4l)$$

$$\Rightarrow \log Z \geq \max_{\mu_i, \mu_j} \left(\sum_i \mu_i b_i + \sum_j \mu_j c_j + \sum_{i,j} w_{ij} \mu_i \mu_j - \sum_i \mu_i \log(\mu_i) + (1 - \mu_i) \log(1 - \mu_i) - \sum_j \mu_j \log(\mu_j) - (1 - \mu_j) \log(1 - \mu_j) \right) \quad (4m)$$

- (d) (3 points) Derive the Loopy BP updates of this RBM model, again assuming a Bethe Cluster Graph (Factor Graph) representation. Express the maximized FFEF (Factored Free Energy Functional, aka Bethe free energy approximation) in terms of the converged Loopy BP messages. While this FFEF is not a theoretical lower bound for Z , argue heuristically when it can be used to approximate Z .

Solution: Assuming a factor graph representation over h_i, v_i and factors a , the messages would be as follows:

$$m_{v_i \rightarrow a} = \prod_{c \in \text{Nbrs}(v_i) \setminus a} m_{c \rightarrow v_i}(v_i) \quad (5a)$$

$$m_{h_i \rightarrow a} = \prod_{c \in \text{Nbrs}(h_i) \setminus a} m_{c \rightarrow h_i}(h_i) \quad (5b)$$

$$m_{a \rightarrow v_i} = \sum_{h_i} \psi(v_i, h_i) \prod_{h \in \text{Nbrs}(a) \setminus v_i} m_{h \rightarrow a}(h) \quad (5c)$$

$$m_{a \rightarrow h_i} = \sum_{v_i} \psi(v_i, h_i) \prod_{v \in \text{Nbrs}(a) \setminus h_i} m_{v \rightarrow a}(v) \quad (5d)$$

The marginals at the nodes:

$$\beta_i(h_i) = \prod_{c \in \text{Nbrs}(h_i)} m_{c \rightarrow h_i}(h_i) \quad (5e)$$

$$\beta_i(v_i) = \prod_{c \in \text{Nbrs}(v_i)} m_{c \rightarrow v_i}(v_i) \quad (5f)$$

Substituting $\psi(v_i, h_i) = \exp(W_{ij} h_i v_j + b_i h_i + c_j v_j)$ in the above equations,

$$m_{a \rightarrow v_i} = \sum_{h_i} \exp(W_{ij} h_i v_j + b_i h_i + c_j v_j) \prod_{h \in \text{Nbrs}(a) \setminus v_i} m_{h \rightarrow a}(h) \quad (5g)$$

$$= [\exp(W_{ij} v_j + b_i + c_j v_j) + \exp(c_j v_j)] \prod_{h \in \text{Nbrs}(a) \setminus v_i} m_{h \rightarrow a}(h) \quad (5h)$$

$$= \exp(c_j v_j) [\exp(W_{ij} v_j + b_i) + 1] \prod_{h \in \text{Nbrs}(a) \setminus v_i} m_{h \rightarrow a}(h) \quad (5i)$$

Similarly,

$$m_{a \rightarrow h_i} = \exp(b_i h_i) [\exp(W_{ij} h_i + c_j) + 1] \prod_{v \in \text{Nbrs}(a) \setminus h_i} m_{v \rightarrow a}(v) \quad (5j)$$

3. (14 points) [DIRICHLET TO OUR RESCUE] We would like to survey the rapidly accumulating research literature on the ongoing COVID-19 outbreak (for instance, a corpus dataset called [CORD-19](#) collects 50,000+ scholarly articles about COVID-19), so as to identify and potentially address gaps in our current knowledge about the virus. To save time, you only want to have a glance at important topics and words in an article. You decided to develop a bot called Dirichlet() that can summarize you the topics present in a research article (along with the important words in varying proportions describing that topic), with no prior knowledge about what topics to look for (which suits us well as someone new to virology or epidemiology). Dirichlet() uses Latent Dirichlet Allocation (LDA) on the whole corpus of articles/documents to achieve this topic modelling task. This question is only on the theory of approximate inference in LDA (though in normal course offerings, we would also have you implement LDA using tools like gensim to analyse a corpus; interestingly, results from LDA analysis of a COVID-19 corpus is depicted [here](#)).

The LDA model can be viewed as a BN with these (local) conditional probabilities:

$$\phi_k \sim \text{Dirichlet}(\beta) \quad (6)$$

$$\theta_i \sim \text{Dirichlet}(\alpha) \quad (7)$$

$$z_{ji} \mid \theta_i \sim \text{Categorical}(\theta_i) \quad (8)$$

$$d_{ji} \mid z_{ji}, \phi_{z_{ji}} \sim \text{Categorical}(\phi_{z_{ji}}) \quad (9)$$

Here j is the index for words ($d_i = \{d_{1i}, \dots, d_{N_i i}\}$), i is the index for documents, and k is the index for topics. Also, we use the following notation: $N_{wki} = |\{j : d_{ji} = w, z_{ji} = k\}|$ (total number of times the word w is assigned to the topic k in document i), $N_{ki} = \sum_{w \in \text{Vocabulary}} N_{wki}$, and $N_{wk} = \sum_i N_{wki}$. We use superscript $(-ji)$ (e.g. N_{wki}^{-ji}) to indicate the corresponding word d_{ji} in document i is not

counted in N_{wki} . By the BN chain rule, the joint distribution is thus given by:

$$\begin{aligned} P(d, z, \theta, \phi | \alpha, \beta) &= \left(\prod_k P(\phi_k | \beta) \right) \prod_i P(\theta_i | \alpha) \prod_j P(z_{ji} | \theta_i) P(d_{ji} | z_{ji}, \phi) \\ &= \left(\prod_k \frac{1}{Z(\beta)} \prod_w \phi_{kw}^{\beta_w - 1} \right) \prod_i \left(\frac{1}{Z(\alpha)} \prod_k \theta_{ik}^{\alpha_k - 1} \right) \prod_j \theta_{i,z_{ji}} \phi_{z_{ji}, d_{ji}}, \end{aligned}$$

where $Z(\alpha)$ (or $Z(\beta)$) denotes the normalizing constant of the corresponding Dirichlet distribution (based on $\Gamma(\cdot)$ functions). Assume the hyperparameters α, β are fixed and known.

- (a) (3 points) Write down $P(d | z, \beta)$ and $P(z | \alpha)$ in terms of functions of $\{N_{wk}, \beta\}$ and $\{N_{ki}, \alpha\}$ respectively. (Hint: Integrate out ϕ and θ respectively; you may find [Section 4 of this writeup](#) on Dirichlet-Categorical compound distbn. helpful.)

Solution:

$$P(d, z, \theta, \phi | \alpha, \beta) = \left(\prod_k P(\phi_k | \beta) \right) \prod_i P(\theta_i | \alpha) \prod_j P(z_{ji} | \theta_i) P(d_{ji} | z_{ji}, \phi) \quad (10a)$$

$$P(d, z | \alpha, \beta) = \int \int P(d, z, \theta, \phi | \alpha, \beta) d\theta d\phi \quad (10b)$$

$$= \int P(z | \theta) P(\theta | \alpha) d\theta \int P(\phi | \beta) P(d | z, \phi) d\phi \quad (10c)$$

Consider,

$$P(z | \alpha) = \int P(z | \theta) P(\theta | \alpha) d\theta = \int \prod_i \frac{1}{Z(\alpha)} \prod_k \theta_{ik}^{\alpha_k - 1} \prod_j \theta_{i,z_{ji}} \quad (10d)$$

$$= \int \prod_i \frac{1}{Z(\alpha)} \prod_k \theta_{ik}^{\alpha_k - 1} \theta_{i,z_{ji}}^{N_{ki}} d\theta \quad (10e)$$

$$= \prod_i \frac{1}{Z(\alpha)} \int \prod_k \theta_{ik}^{N_{ki} + \alpha_k - 1} d\theta \quad (10f)$$

Now, integral over the simplex,

$$\int \frac{1}{Z(\alpha)} \prod_k \theta_{ik}^{\alpha_k - 1} d\theta = 1 \implies \int \prod_k \theta_{ik}^{\alpha_k - 1} d\theta = Z(\alpha) \implies \int \prod_k \theta_{ik}^{N_{ki} + \alpha_k - 1} d\theta = Z(N_{ki} + \alpha) \quad (10g)$$

$$\therefore P(z | \alpha) = \prod_i \frac{Z(N_{ki} + \alpha)}{Z(\alpha)} \quad (10h)$$

$$P(d | z, \beta) = \int P(\phi | \beta) P(d | z, \phi) d\phi \quad (10i)$$

$$= \int \prod_k \frac{1}{Z(\beta)} \prod_w \phi_{kw}^{\beta_w - 1} \prod_j \phi_{z_{ji}, d_{ji}} d\phi \quad (10j)$$

$$= \int \prod_k \frac{1}{Z(\beta)} \prod_w \phi_{kw}^{\beta_w - 1} \prod_j \phi_{z_{ji}, d_{ji}} d\phi \quad (10k)$$

$$= \int \prod_k \frac{1}{Z(\beta)} \prod_w \phi_{kw}^{\beta_w - 1} \phi_{z_{jk}, d_{jk}}^{N_{wk}} d\phi \quad (10l)$$

$$= \prod_k \int \frac{1}{Z(\beta)} \prod_w \phi_{kw}^{N_{wk} + \beta_w - 1} d\phi \quad (10m)$$

$$= \prod_k \frac{Z(N_{kw} + \beta)}{Z(\beta)} \quad (10n)$$

Also,

$$P(d, z | \alpha, \beta) = \left[\prod_i \frac{Z(N_{ki} + \alpha)}{Z(\alpha)} \right] \left[\prod_k \frac{Z(N_{kw} + \beta)}{Z(\beta)} \right] \quad (11)$$

(b) (1 point) Exact probabilistic inference on $P(z | d)$ is infeasible – explain why.

Solution: The main quantity of interest is the posterior distribution of the hidden variables given the observed data:

$$P(\theta, \phi, z | d, \alpha, \beta) = \frac{P(P(\theta, \phi, z, d | \alpha, \beta))}{P(d | \alpha, \beta)} \quad (12)$$

However, the marginal in the denominator cannot be computed exactly and hence the distribution is intractable to compute.

More intuitively, selection of the topic given a word is not possible, since a subset of words can be present in many topics.

- (c) (5 points) Since exact inference is infeasible, we will use approximate inference. In particular, we are first interested in “collapsed” Gibbs sampling (“collapsed” since ϕ and θ are integrated out in the inference procedure). Prove the following LDA collapsed Gibbs sampling equation:

$$p(z_{ji} = k \mid z \setminus z_{ji}, d, \alpha, \beta) \propto (N_{ki}^{(-ji)} + \alpha_k) \frac{N_{wk}^{(-ji)} + \beta_w}{N_k^{(-ji)} + \sum_{w'} \beta_{w'}}$$

where $w = d_{ji}$.

(Hint: $\Gamma(x + 1) = x\Gamma(x)$)

Solution: For Gibbs sampling, we want to compute the probability of a topic z being assigned to a word w_j , given the topic assignments to all other words i.e. we need to find $p(z_t \mid Z_{-t}, \alpha, \beta, W)$.

Here, the subscript t indicates a token $\{j', i'\}$ i.e. a word j' in document i' .

The Gibbs sampling equation can be derived as follows:

$$P(z_t = k' \mid Z_{-t}, W) = \frac{P(Z, W)}{P(Z_{-i}, W)} \quad (13a)$$

$$= \frac{P(W \mid Z)P(Z)}{P(W_{-t} \mid Z_{-t})P(Z_{-i})P(w_i)} \propto \frac{P(W \mid Z)P(Z)}{P(W_{-i} \mid Z_{-i})P(Z_{-i})} \quad (13b)$$

$$= \frac{\prod_i Z(N_{ki} + \alpha) \prod_k Z(N_{kw} + \beta)}{\prod_i Z(N_{ki}^{(-t)} + \alpha) \prod_k Z(N_{kw}^{(-t)} + \beta)} \quad (13c)$$

$$= \frac{\prod_i Z(N_{ki} + \alpha) \prod_k Z(N_{kw} + \beta)}{\prod_i Z(N_{ki}^{(-ji)} + \alpha) \prod_k Z(N_{kw}^{(-ji)} + \beta)} \quad (13d)$$

Consider the first term $\frac{\prod_i Z(N_{ki} + \alpha)}{\prod_i Z(N_{ki}^{(-ji)} + \alpha)}$

$$\frac{\prod_i Z(N_{ki} + \alpha)}{\prod_k Z(N_{ki}^{(-ji)} + \alpha)} = \frac{\prod_i \prod_k \Gamma(N_{ki} + \alpha)}{\Gamma \sum_k (N_{ki} + \alpha)} \frac{\Gamma \sum_k (N_{ki}^{(-ji)} + \alpha)}{\prod_i \prod_k \Gamma(N_{ki}^{(-ji)} + \alpha)} \quad (13e)$$

$$= \frac{\prod_i \prod_k \Gamma(N_{ki} + \alpha)}{\prod_i \prod_k \Gamma(N_{ki}^{\neg ji} + \alpha)} \frac{\Gamma \sum_k (N_{ki}^{\neg ji} + \alpha)}{\Gamma \sum_k (N_{ki} + \alpha)} \quad (13f)$$

Now, given a token/word $w_i = \{j', i'\}$ whose latent topic assignment is k' ,

$$N_{ki} = \begin{cases} N_{ki}^{\neg ji} & \text{for } k \neq k' \\ N_{ki}^{\neg ji} + 1 & \text{for } k = k' \end{cases} \quad (13g)$$

$$\therefore \frac{\prod_i Z(N_{ki} + \alpha)}{\prod_i Z(N_{ki}^{\neg ji} + \alpha)} = \frac{\prod_i \Gamma(N_{k'i} + \alpha)}{\prod_i \Gamma(N_{k'i}^{\neg ji} + \alpha)} \frac{\Gamma(\sum_{k \setminus k'} (N_{ki}^{\neg ji} + \alpha) + (N_{k'i}^{\neg ji} + \alpha))}{\Gamma(\sum_{k \setminus k'} (N_{ki} + \alpha) + (N_{k'i} + \alpha))} \quad (13h)$$

Using Equation 13g,

$$\frac{\prod_i Z(N_{ki} + \alpha)}{\prod_i Z(N_{ki}^{\neg ji} + \alpha)} = \frac{\prod_i \Gamma(N_{k'i}^{\neg ji} + 1 + \alpha)}{\prod_i \Gamma(N_{k'i}^{\neg ji} + \alpha)} \frac{\Gamma(\sum_{k \setminus k'} (N_{ki}^{\neg ji} + \alpha) + (N_{k'i}^{\neg ji} + \alpha))}{\Gamma(\sum_{k \setminus k'} (N_{ki}^{\neg ji} + \alpha) + (N_{k'i}^{\neg ji} + 1 + \alpha))} \quad (13i)$$

Now, using $\Gamma(x + 1) = x\Gamma(x)$, we get:

$$\frac{\prod_i Z(N_{ki} + \alpha)}{\prod_i Z(N_{ki}^{\neg ji} + \alpha)} = \frac{(N_{k'i}^{\neg ji} + \alpha)}{\sum_k (N_{k'i}^{\neg ji} + \alpha)} \quad (13j)$$

Similarly,

$$\frac{\prod_i Z(N_{kw} + \beta)}{\prod_i Z(N_{ki}^{\neg ji} + \beta)} = \frac{(N_{k'i}^{\neg ji} + \beta)}{\sum_w (N_{k'w}^{\neg ji} + \beta)} \quad (13k)$$

$$\therefore P(z_t = k \mid Z_{-t}, W) = \frac{(N_{ki}^{\neg ji} + \alpha)}{\sum_k (N_{ki}^{\neg ji} + \alpha)} \frac{(N_{k'i}^{\neg ji} + \beta)}{\sum_w (N_{k'w}^{\neg ji} + \beta)} \quad (13l)$$

$$\therefore P(z_t = k \mid Z_{-t}, W) \propto \sum_k (N_{ki}^{ji} + \alpha_k) \frac{(N_{ki}^{ji} + \beta)}{\sum_w (N_{kw}^{ji} + \beta_w)} \quad (13m)$$

- (d) (5 points) As an alternate approximate inference approach, we will use a variational method (in particular, Naïve Mean Field or NMF, again “(θ, φ)-collapsed”) that approximates $P(z \mid d)$ using a fully factored distbn.:

$$Q(z \mid \lambda) = \prod_i \prod_j Q_{ji}(z_{ji} \mid \lambda_{ji}),$$

where $Q(z_{ji} = k \mid \lambda) = Q_{ji}(z_{ji} = k \mid \lambda_{ji}) := \lambda_{jik}$. The free variational parameters $\lambda = \{\lambda_{jik}\}$ that minimizes the KL-divergence $D(Q \parallel P)$ can be heuristically found using an iterative fixed-point NMF theorem seen in class. Apply this theorem to derive this NMF update rule:

$$Q_{ji}(z_{ji} = k) := \lambda_{jik} \propto (E_{Q_{-ji}}[N_{ki}^{(-ji)}] + \alpha_k) \frac{E_{Q_{-ji}}[N_{wk}^{(-ji)}] + \beta_w}{E_{Q_{-ji}}[N_k^{(-ji)}] + \sum_{w'} \beta_{w'}}$$

where $w = d_{ji}$ and Q_{-ji} refers to the (fully-factored) distribution over all z variables other than z_{ji} using the current iteration settings of all λ parameters other than λ_{jik} .

(Hint: You may use, $\log \frac{\Gamma(x+q)}{\Gamma(x)} = \sum_{p=0}^{q-1} \log(x+p)$, for real $x > 0$ and integer $q > 0$; and a crude approximation, $E[\log(X)] \simeq \log(E[X])$.)

Solution: The approximating distribution,

$$Q(z \mid \lambda) = \prod_i \prod_j Q_{ji}(z_{ji} \mid \lambda_{ji})$$

We want to minimize $KL(Q \parallel P)$,

$$KL(Q \parallel P) = E_Q \log \left(\frac{Q(z \mid \lambda)}{P(z, d, \theta, \phi \mid \alpha, \beta)} \right) \quad (14a)$$

$$= E_Q \left(\log Q(z \mid \lambda) - \log P(z, d, \theta, \phi \mid \alpha, \beta) \right) \quad (14b)$$

$$= E_Q \left(\log Q(z \mid \lambda) - \log P(z, d \mid \theta, \phi, \alpha, \beta) + \log P(\theta, \phi \mid \alpha, \beta) \right) \quad (14c)$$

$$= E_Q \left(\log Q(z | \lambda) - \log P(z | d, \alpha, \beta) \right) + \log P(\theta, \phi | \alpha, \beta) \quad (14d)$$

Thus,

$$KL(Q \parallel P) = F + \log P(\theta, \phi | \alpha, \beta) \quad (14e)$$

To minimize $KL(Q \parallel P)$, we can thus minimize F .

$$F = E_Q \left(\log Q(z | \lambda) - \log P(z, d | \alpha, \beta) \right) \quad (14f)$$

$$= E_Q \left(\log Q(z | \lambda) - \log P(z, d | \alpha, \beta) \right) \quad (14g)$$

Minimizing this w.r.t λ_{jik} , we get:

$$\lambda_{jik} = \frac{\exp \left(E_{Q(Z_{-ij})} P(z_{ij} = k, Z_{-ij}, d | \alpha, \beta) \right)}{\sum_{k'} \exp \left(E_{Q(Z_{-ij})} P(z_{ij} = k', Z_{-ij}, d | \alpha, \beta) \right)} \quad (14h)$$

Plugging in Equation 11 for $P(z, d | \alpha, \beta)$, substituting $z_{ij} = k$, and using $\log \frac{\Gamma(x+q)}{\Gamma(x)} = \sum_{p=0}^{q-1} \log(x+p)$ we get

$$\lambda_{jik} = \frac{\exp \left(E_{Q(Z_{-ij})} \left[\log(N_{ki}^{-ji} + \alpha) + \log(N_{kw}^{-ji} + \beta) - \log(\sum_w N_{kw}^{-ji} + \beta_w) \right] \right)}{\sum_{k'} \exp \left(E_{Q(Z_{-ij})} \left[\log(N_{ki}^{-ji} + \alpha) + \log(N_{kw}^{-ji} + \beta) - \log(\sum_w N_{kw}^{-ji} + \beta_w) \right] \right)} \quad (14i)$$

Using $E[\log(X)] \simeq \log(E[X])$,

$$E_{Q(z_{-ij})} \log(N_{ki}^{-ji} + \alpha) = \log(E_{Q(z_{-ij})} [N_{ki}^{-ji} + \alpha]) = \log(E_{Q(z_{-ij})} [N_{ki}^{-ji}] + \alpha) \quad (14j)$$

$$\Rightarrow \exp \left(E_{Q(z_{-ij})} \log(N_{ki}^{-ji} + \alpha) \right) = \exp \left(\log(E_{Q(z_{-ij})} [N_{ki}^{-ji}] + \alpha) \right) = E_{Q(z_{-ij})} [N_{ki}^{-ji}] + \alpha \quad (14k)$$

$$\therefore \lambda_{jik} \propto \exp \left(\log \left[(E_{Q_{-ji}} [N_{ki}^{-ji}] + \alpha_k) \frac{E_{Q_{-ji}} [N_{wk}^{-ji}] + \beta_w}{E_{Q_{-ji}} [N_k^{-ji}] + \sum_{w'} \beta_{w'}} \right] \right) \quad (14l)$$

$$\therefore \lambda_{jik} \propto (E_{Q_{-ji}} [N_{ki}^{-ji}] + \alpha_k) \frac{E_{Q_{-ji}} [N_{wk}^{-ji}] + \beta_w}{E_{Q_{-ji}} [N_k^{-ji}] + \sum_{w'} \beta_{w'}} \quad (14m)$$

- (e) (0 points) (*Optional Intuition/Background Question*) A key inference query in a latent model like LDA is the conditional (posterior) probability of the hidden variables, viz., $P(z, \theta, \phi \mid d, \alpha, \beta)$, but we focused only on the “ (θ, ϕ) -collapsed” probability $P(z \mid d, \alpha, \beta)$ so far, since collapsed approximate inference (collapsed Gibbs sampling or collapsed variational inference) methods are often more accurate than full (non-collapsed) approximate inference. Furthermore, once we’ve information on z , we can get information about (θ, ϕ) as described below.

Estimation of θ_i (document-topic proportion) and ϕ_k (topic-word distribution) are greatly simplified by knowing z_{ji} (topic assignment for each word d_{ji} in document i , drawn from $P(z \mid d)$ given by the collapsed Gibbs sampler above or from $Q(z)$ given by the collapsed variational method above). **Write down the most intuitive formula you can think for estimating $\theta_i = \{\theta_{ik}\}$ and $\phi_k = \{\phi_{kw}\}$ from all z_{ji} .** Argue why these intuitive point estimates are in fact the posterior mean of θ_i and ϕ_k (the posterior here refers to the conditional distribution $(\theta_i \mid z, d)$ or equivalently $(\theta_i \mid z)$ for θ_i , and a similar conditional distribution $(\phi_k \mid z, d)$ for ϕ_k)).

(Note: Drawing the graph structure of all three models in this assignment can aid your intuition; you can use a grid-graph for image denoising, a complete bipartite graph for RBM, and a plate network diagram for LDA.)